Verifiable Multi-Agent RL Task Construction from Information Seams and Permission Partitions

A method for turning an agentic task database into multi-agent RL tasks, instantiated on AppWorld

Jinao Wang
Multi-Agent Task Forge
17 July 2026

Abstract This report presents a method, and one substrate carried through it end to end. The method takes an agentic task database that was never meant for multi-agent work and turns its single-agent tasks into training tasks that cannot be finished without collaboration — without writing a new task, a new environment, or a new verifier. It asks four things of a database and nothing else: a verifier somebody else wrote, an installable world, a reference solution per task, and an action surface that divides along role boundaries. Any database clearing those admits the method; AppWorld is the one this report runs, and §6.6 states plainly that it is the only one. Concretely, on AppWorld: read the 147 tasks that carry ground truth, parse their reference solutions, and keep, via one forward taint pass, the 39 whose facts genuinely cross an app boundary. The roster comes from the apps the reference solution actually touched, rather than from padding a task out to look multi-agent. Each task is then wrapped in a permission partition: the Main agent holds no app API at all and can only delegate, while the verifier and the ground truth stay on a side it cannot reach. Whether the task is done is still decided by AppWorld's own database-state checks, and the reward is the fraction of state requirements that pass. No LLM judge is involved.

The cost structure of this method is lopsided, and that is what determines how it scales. Selecting tasks, deriving rosters, and rendering task directories are all static analysis with no model calls: spreading 39 tasks into 78 Harbor task directories is one command and a few minutes. The bills come afterwards. A task can only ship if a reference solution proves it solvable, and those are currently written by hand; a cell only counts once enough seeds have been run to show whether it produces a non-degenerate training signal. So the way up is not to keep adding knobs. It is to automate reference-solution generation first (§6.3 — the seam analysis has already computed its skeleton), then to widen the source pool: the ceiling on this dimension is 13 independent task families, not 39 tasks, and certainly not 78 directories.

Contents

1 Problem and Goal

A task usable for reinforcement learning needs at least three parts: the instruction the model receives, the environment it can interact with, and the reward issued once the episode ends. Single-agent tasks are usually easy to verify — check whether a file was created, whether a database row changed, whether the tests pass. Multi-agent tasks are harder, because we care about more than the end result: we want the model to hand subtasks to the right role and to move the necessary facts between roles.

There is an easy mistake here. Add a few roles and a group chat, and the task looks multi-agent. But if the Main agent still holds every tool, or if any sub-agent could finish the task alone, then the roles are set dressing and produce no new learning signal. Score the transcript on whether it *looks* collaborative instead, and the reward becomes subjective and impossible to scale.

So this project narrows the goal: start from real tasks that already have a trustworthy verifier, then use permission and information constraints to make collaboration *necessary* for finishing. The model must demonstrate coordination, but the reward is still decided solely by the end state.

Stated at the level it is meant to be reused at, that goal is a **method** rather than a dataset. No corpus of multi-agent traces with environments and free verifiers exists, so anyone building this data must manufacture the multi-agent structure themselves; the only real decision is *which layer* they manufacture. Manufacture the verifier and the mistakes become invisible, because your own tests will agree with your own errors — §4.1 is that failure, measured. Manufacture only *constraints*, which change who may do what and never what counts as done, and the failure mode becomes a task that is too easy or too hard: visible, and priceable by the criterion of §4.3. The rest of this report is that method in four moves — admit a database (§6.6), mine its seams (§3.1), constrain access (§3), accept only the cells that land between the floor and the control (§4.3) — and one database, AppWorld, carried through all four.

2 Capability Selection and Learning Order

Multi-agent systems involve many capabilities. To keep the training signal legible, this design first picks the ones that can be verified from environment outcomes and that compose in layers. MAST groups common failures into system design, inter-agent misalignment, task verification and termination, and related classes [4]; MultiAgentBench shows that star, chain, tree and graph topologies impose different coordination demands [3]. At the data-construction stage, though, changing every factor at once is the wrong move.

Table 1: Capabilities currently prioritised for training

Capability	What the model must learn	How failure shows up
Decomposition and dependency recognition	Break a goal into executable subtasks and work out which intermediate results must come first	A necessary app is skipped, the order is wrong, a downstream step lacks its upstream fact
Role assignment	Route each subtask to the role whose information and tools fit it	The request goes to the wrong role, or a role is asked to read what it cannot see
Cross-role communication	Carry entities, date ranges and prior results into the delegation, so local execution serves the global goal	The sub-agent satisfies its own request, but the final world state is still wrong
Parallel scheduling	Cut needless serial waiting while respecting dependencies and resource limits	Resource conflicts, dropped tasks, unmet dependencies, or an inflated makespan

The first three capabilities in Table 1 are exercised directly by the current delivery. Parallel scheduling is the exception. It is listed because it belongs to the same family — verifiable from environment state — but this AppWorld-based delivery does not cover it: the sidecar is single-threaded (AppWorld's world is not thread-safe), delegation calls block serially, and the terminal scalar $r = N_{\text{pass}}/N$ has no way to price makespan. Training it needs a runtime that can express concurrency, which is out of scope here — the same reason Theory of Mind is not listed as its own dimension below.

These capabilities suit layered training. The first layer asks only that the model identify the roles and delegate once, correctly. The second adds a cross-app dependency, so the Main must carry an upstream result downstream. The third hides the roles' capability documentation, forcing the model to work out who might know what. Only later come delegation budgets, more roles, and richer topologies. One clarification is due here: the third layer already implies Theory of Mind — judging *who might know what* is exactly modelling another role's knowledge state, and the failure criterion for "cross-role communication" in Table 1 is the same thing seen from the other side, where the Main assumes the sub-agent knows as much as it does and issues a delegation the sub-agent cannot act on. The reason Theory of Mind and dynamic recovery are still not separate dimensions is not that they are unimportant. It is that the current reward is a single terminal scalar and cannot attribute one failure to that link alone.

That limitation belongs to this design, not to the capability. By the rule of §4.1 — the sin is the unmeasured verifier, not the language model — attribution is not hopeless: a teacher model that reads the ledger and decides whether a failure lay in decomposition, routing or communication is entirely buildable. It is a self-designed judge, so it needs a validation study (agreement against human labels), not a prohibition. What is unacceptable is using one *without measuring it*, which is precisely what v1 did with that one line of division. So the route is: the terminal scalar stays the only training reward, and the teacher model is used for *diagnosis* and curriculum design; once its attribution accuracy has been measured, we can discuss whether it may enter the reward. This report has not done that step, so the table above lists only terminally verifiable capabilities.

> The suggested progression is: single-shot role routing → star + visible capability docs → star + role names only → limited budget and multi-level dependencies → chain/tree and runtime faults.

The current implementation and experiments reach the third layer. The last two have a clear construction path, but each must be shown solvable and shown to add difficulty before it enters a training set.

3 From Single-Agent Task to Collaborative Task

The method fits in one sentence: *keep the task and the verifier; restructure only how agents reach the task.*

The original task already has a user instruction, an interactive environment, and a programmatic verifier. Forging adds coordination constraints — communication topology, role visibility, delegation budget — in a layer outside it. However the model decomposes and communicates, the original environment tests still decide whether the task is done.

More precisely, write the original task as

$$\tau = (x, \mathcal{E}, J), \qquad \mathcal{E}_c = W(\mathcal{E}; c), \qquad \tau_c = (x, \mathcal{E}_c, J).$$

Here x is the user instruction, $\mathcal{E}$ the environment, J the programmatic verifier. W is a wrapping operator that acts *only on the environment*: it does not rewrite the instruction and does not rewrite the verifier — x and J are literally identical on both sides. That is the entire source of this method's safety, and the only thing separating it from "design your own verifier". The constraint

$$c = (P, V, G, B)$$

has four components: P, the permission partition (which role holds which app's API); V, role visibility (what the Main may know about the roles); G, the communication graph (who may message whom); and B, the delegation budget.

Three notes. First, P refers to the *structure* of the partition — the Main holds no API, each specialist binds to exactly one app. Which apps those are is not a knob; it is determined by the reference solution (§3.1). Second, P is the most fundamental of the four: without it the Main can bypass coordination and simply do the task, and the other three become moot. Third, J is unchanged word for word, but what it reads is the terminal state inside the sidecar; W simultaneously guarantees that neither J nor the ground truth exists anywhere on the agent's filesystem (§5.1).

This is close in spirit to SWE-smith's principle of extending software tasks with the tests that already exist [2]: reuse verification logic that is already trustworthy rather than designing fresh judging logic per task. The whole forging process reduces to three steps: identify cross-app dependencies in real tasks, derive the roster from those dependencies, and wrap the same task into different coordination configurations.

3.1 Selecting tasks from real dependencies

AppWorld provides 9 everyday apps, 457 APIs and 750 complex tasks (the currently published dataset actually contains 732), and uses database-state tests to decide both whether a task was completed and whether it caused unintended modifications [1]. This repository reads the 147 tasks in train and dev that carry full ground truth and analyses which apps their reference solutions actually call.

The roster is decided entirely by the task itself. If a task's reference solution calls both phone and venmo, it naturally corresponds to two specialists. `supervisor.complete_task`, used to submit the answer, does not count as a role, because it supplies no business information. Under that rule, 51 of the 147 tasks genuinely need at least two apps; the rest are discarded outright rather than padded with an irrelevant role to manufacture a multi-agent shape.

3.2 Turning a single-agent task into a collaborative one

The wrapped environment holds one Main agent and several app-bound sub-agents. The Main sees the user goal but holds no app API; the phone specialist can reach only phone, the venmo specialist only venmo. Sub-agents know only the local request the Main sent them, never the whole task.

Take one task from the current delivery: "On my Venmo social feed, like all of today's transactions involving my roommates." The Main must first ask phone which contacts are roommates, then hand that list of names, together with the date range, to venmo. Neither sub-agent can finish the task alone: phone knows the relationships but cannot like anything; venmo can like transactions but does not know who the roommates are. The task therefore carries a clean dependency chain of its own:

phone yields roommate names $\longrightarrow$ Main carries names and range $\longrightarrow$ venmo iterates and acts.

This construction tests the model along three axes at once: discovering which role holds which information, ordering the subtasks correctly, and carrying the context the downstream role actually needs. But the terminal reward can only judge whether the whole thing got done; it cannot attribute one failure to one of those axes (see §8.1). The prompt remains a natural-language task, and the dependency graph exists only in the environment and the reference solution — it is never shown to the model.

4 The Forging Method: Four Elements

SWE-smith's leverage is not clever bug injection. It is that it never designs a verifier: break a repository that used to work, and let the repository's own tests decide whether the fix is right [2]. Taken apart, the method has only four elements: an invariant (the repo's tests), a set of operators

(five ways to produce candidates), a criterion (the patch counts only if tests that used to pass go red), and a measured yield for each operator (56%, 35%, 40.2%, 33.8%, 96.9%).

Any forging method must answer the same four questions. This section places the present design against those four elements and names the gaps.

4.1 Invariant: inherit the verifier; if you must build one, measure it first

Forging can act at two levels: design the world and the verifier, or design only the constraints. The failure modes are not symmetric. Choose the wrong constraint and the task is too easy or too hard — which is measurable. Design the verifier wrong and the reward measures the wrong thing — which is invisible, because your own tests will agree with your own mistake. This design therefore touches only the latter: constraints change *who may do what* and never touch *what counts as done*. That is not laziness; it trades an invisible failure mode for a visible one.

The principle targets *unmeasured* self-designed verifiers, not any particular technique. Language models have several defensible positions in a forging pipeline: **generating candidates** (three of SWE-smith's five operators work this way, and the issue text is model-generated too); **playing the environment** (the specialists here are real models — swap in a scripted executor and the question "did the Main state its request clearly?" stops existing); and **acting as teacher where no free verifier exists**. The third needs a validation study attached: compare its judgements against a small batch of human labels and report agreement. This is not a special demand on language models — every self-designed verifier owes the same thing, including the information-seam analysis this report uses in §4.4 to filter tasks, whose error rate is likewise unmeasured.

A free verifier therefore stays the first choice, not because the alternatives are unworkable, but because that validation study is a real cost, and inheriting a verifier somebody else already validated avoids paying it.

Choosing a substrate. The invariant comes from the substrate, so choosing the substrate is the method's first step. A usable substrate must: ship a programmatic verifier (*strongly* preferred, since it is free and already validated by others); be real, installable and executable; carry ground truth on at least the train split, or you are left guessing which tasks qualify; and have an action surface divisible along at least two role boundaries.

A fifth condition came out of practice on this project: the seam must be an information seam, not an operation seam. If the two sides can each work independently and merely be sequenced, that is parallel division of labour — no fact has to cross, and adding roles is pure form. Only when the action on one side depends on a fact obtainable only from the other do delegation and communication produce a learning signal. This criterion also explains why software-repository substrates are a poor fit: they share one filesystem, no fact is private, and any partition degenerates into "paste the file across".

4.2 Operators: how candidates are produced

Operators fall into three classes by how they produce candidates; see Table 2.

Table 2: Forging operators, classified by how candidates are produced

Class	SWE-smith analogue	This design's operator	Task or rules	Yield	Status
Procedural	Procedural (13 AST transforms)	`partition`: split roles by the apps the reference solution touches; the Main holds no API	rules	**39/51** selection; 3/3 cells	shipped
Procedural	Procedural	`blind`: hide the roles' capability docs, leaving only names	rules	**2/3** cells	shipped
Procedural	Procedural	`route` (delegation budget): cap the number of delegations	rules	unmeasured	built, unshipped
Procedural	Procedural	`route` (chain/tree topology): change who may talk to whom	rules	0/3 cells	not built
Procedural	Procedural	`perturb`: withdraw a role mid-run, forcing replanning	task	—	not built
LM rewrite	LM Modify / LM Rewrite	`weaken`: strip dates and entities from the instruction into hidden user state in the sidecar	task	—	not built
Combine	Combine	`combine`: merge two tasks that share an entity	task (+ verifier)	—	not built

In the table, *built, unshipped* means implemented but present in no published configuration; *not built* means there is no code. The delegation budget is the former: it is fully implemented in `partition.py` and enforced on both the Main side and the sidecar side, but both published configurations pass an empty budget. The chain topology is the latter: the specialist-to-specialist handoff is defined in `partition.py` and called by nothing, so tasks under that topology are unsolvable and their difficulty signal is fake. The operator names are this report's; the code exposes `Topology`, `Visibility` and `delegation_budget`. SWE-smith's PR Mirror has no analogue here, because there is no PR to mirror.

One distinction has to be faced squarely: operators that change the rules produce no new tasks, only new configurations. However `partition` and `blind` are combined, the 39 tasks that passed the seam gate remain those same 39; $39 \times 2 = 78$ counts configurations, not tasks. SWE-smith's 50k all come from operators that edit the world itself. To make the task count actually grow, you need operators that modify the task — and that is exactly where yield stops being free.

`combine` is the only one that would require rebuilding the verifier: AppWorld's verifier checks side effects, so task A's "no new venmo transactions" is tripped by task B's legitimate actions, and $J_A \wedge J_B$ is not a self-consistent verifier. It has the largest potential output and sits closest to the very practice this method exists to avoid.

4.3 Criterion: when does a candidate count

SWE-smith's criterion fits in a sentence: apply the patch, run the tests, and the candidate counts if tests that used to pass go red. Mechanical, free, judgement-free.

This design's criterion is a sandwich. Write the do-nothing floor as r_0, the unconstrained control as r_{open}, and a configuration's score as r. The configuration counts only when

$$r_0 < r < r_{\text{open}}$$

and the control is not itself on the floor ($r_{\text{open}} > r_0$). Landing on r_0 means the cell is indistinguishable from doing nothing: the reward cannot tell trying from not trying, and the gradient is zero — whether the cause was a genuinely hard task or a broken harness. Hitting r_{open} means the constraint never bit, and the cell measures the base task rather than coordination. And if r_{open} is itself on the floor, the base task is already broken, so no knob's effect on top of it can

be measured — reading a constrained score against a control that already failed is precisely the path by which a harness fault gets published as difficulty.

Solvability is a *separate* gate and is not part of this criterion: it is proved by actually running the reference solution inside the container, driven by `appworld_oracle_report.py`, with results recorded in `sweep/appworld_oracle.json`.

This rule previously existed only as prose, and this repository published 0.333 as "the partition is hard" for two days — it was the floor. It is now `forge/appworld/validity.py`, and `python -m forge.appworld.cli judge` prints the table above directly.

4.4 Yield: forge ten, keep how many

Yield is the fraction of forged candidates that are actually usable. SWE-smith measured one number per operator and used them to decide where to spend. Forging filters candidates at two points, so there are two yields: **selection** (how many tasks in the pool qualify) and **rendering** (how many of a qualifying task's rendered cells are trainable). Neither had been measured before.

Selection yield: 12 of the 51 are merely parallel

`select.py`'s criterion is `len(roster) >= 2`. That proves "the reference solution touched two apps", while its own docstring claims "a task that cannot be coordinated is dropped, not decorated" — two different statements, with the step between them never verified.

The difference is the one from §4.1: the seam must be an information seam. If two apps each do their own work and merely need ordering, nothing is carried across the seam, and splitting roles is just division of labour. This is measurable, and needs no model calls: parse the reference solution, run a forward taint pass, and check whether a fact obtained from one app is genuinely consumed by another app's action — appearing in its arguments, or in the condition guarding it. `seams.py` does this, with results in `sweep/appworld_seams.json`.

Table 3: Selection yield: three filters

Filter	Kept	Share
naive: count `supervisor.complete_task` as a collaborator	147/147	100%
strict: drop the submit channel (`select.py`'s current rule)	51/147	34.7%
+ **information seam**: require a fact to actually cross	**39/147**	**26.5%**

12 of the 51 have no information seam, and they are not scattered: they are four complete task families of three variants each. Two of those families are `phone+venmo`, the same pair as the shipped family, and still have no seam. Splitting roles on them trains parallel dispatch, not coordination.

This is the same shape as the supervisor trap, one level up: the 100% → 34.7% step removes "a submit channel that carries no information", and the 34.7% → 26.5% step removes "a second app that carries no information". The first step was taken; the second was not — and the price was treating **23.5% decorative tasks as coordinatable ones**.

Worth recording: in this analysis, 15 tasks' seams pass *only* through a guard clause (the action follows an `if ... : continue`), while 36 pass through arguments. All three shipped tasks are of the first kind. An implementation that only checks "does an upstream return value appear in the arguments" would miss every one of them — including the only task family this project has ever shipped.

Rendering yield: of the qualifying tasks, how many rendered cells are trainable

Applying the sandwich criterion of §4.3 to the sweep gives Table 4.

Table 4: The sweep re-read under the sandwich criterion (five seeds per cell)

Configuration	_1	_2	_3	Usable cells	Note
star-docs	**0.833**	**0.833**	**0.900**	3/3	every cell drops below the control somewhere in its five draws
star-names	**0.833**	below floor	**0.333**	2/3	one cell sits on 0.167 for all five replicates
chain-names (not built)	floor	floor	below floor	0/3	a known fake signal; 2 seeds, so not a measured yield

The two shipped configurations cover 6 cells, and 5 land in the useful band. star-docs' mean of 0.855 comes from every cell spreading below the control across its replicates; star-names' 0.445 comes from two cells that bite and one that scores 0.167 on all five draws (§7 explains why a cell can fall below the floor). Every rollout logged `ran=True` with no error, so the one floored cell is a real failure rather than a harness fault — and for RL data it is still unusable: there is no gradient between it and doing nothing.

This table read **1/3 and 0/3** one seed ago, and nothing about the tasks changed in between. At a single draw star-docs tied the control on _1 and _2, so the rule called them NO-BITE — correctly, on the evidence available: a cell that always ties its ceiling teaches nothing. Four more draws put those same cells at 0.667 and 0.833, so min < control and they are valid. The variance was always there; one point cannot exhibit variance, which is precisely what `SEEDS_FOR_YIELD` encodes. The correction runs *upward*, and it is worth noticing that a criterion built to deflate a flattering number deflated an unflattering one just as readily.

One observation deserves its own line: the verdicts did not change between two versions of the sweep, only between two sample sizes. In the previous sweep star-names averaged 0.778; in this one it averages 0.445 — a gap of 0.333, a full third of the range — and the answer to "which cells are trainable" did not move by a single cell. A mean that swings by a third while the conclusion holds still is exactly the argument for judging cell by cell instead of averaging.

Finally, the scope of these numbers. With five replicates in every shipped cell, `SEEDS_FOR_YIELD` is satisfied and the rates above are a yield rather than an observation about the draws in hand. What they are not is a yield for the *method*: all three tasks come from one family, so 0.145 is priced against sampling noise and not against task variation. Turning this into a claim about the method needs more task families, not more seeds and not more configurations.

5 Runtime, Trajectory and Verification

5.1 The container boundary

Harbor packages each task as a standalone unit containing the instruction, the container environment, the tests and a reference solution [5]. The container holding the Main agent exposes only a thin `team` interface; the AppWorld world, the sub-agents and the final state all live on the sidecar. The Main can therefore only advance the task by delegating, and cannot bypass coordination to touch the apps directly.

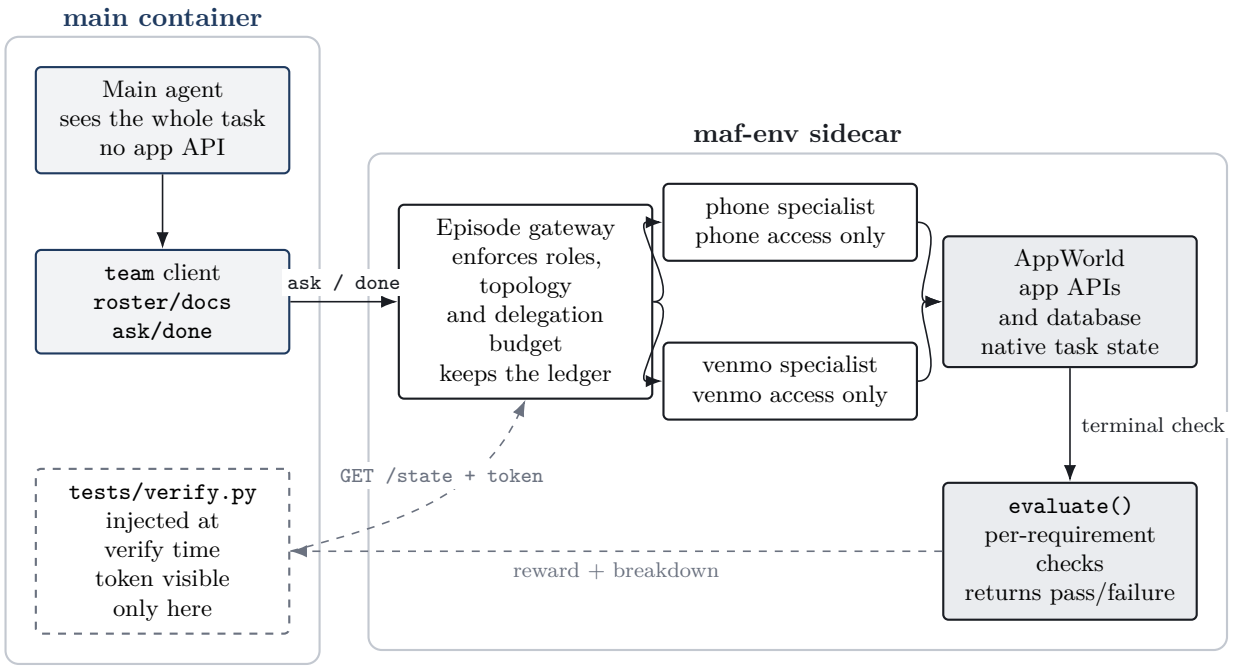

Figure 1: Container architecture of a Harbor task. The solid box on the left is the main container the model can see; AppWorld, the sub-agents, the ledger and the judging logic all stay in the sidecar. Dashed lines mark the verification phase, which happens only after the agent has finished.

Two boundaries in Figure 1 matter most. First, the Main's filesystem holds no AppWorld, no ground truth and no verifier; it can only issue natural-language delegations through `team`. Second, the verifier token never exists during the agent phase; Harbor injects the tests into the main container only after execution ends, and the verifier reads the terminal state from the sidecar. Role constraints are thus enforced by the environment, and the model cannot read its own reward mid-run.

5.2 How a task runs

Figure 2 shows the reference trajectory for task `2a163ab_1`. Each `team ask` in the figure may contain several rounds of sub-agent reasoning and API calls internally; only cross-role information flow is kept here. What decides success is not the number of messages but whether the Main obtains the roommate list first, then hands venmo the names, the date range and the full-pagination requirement together.

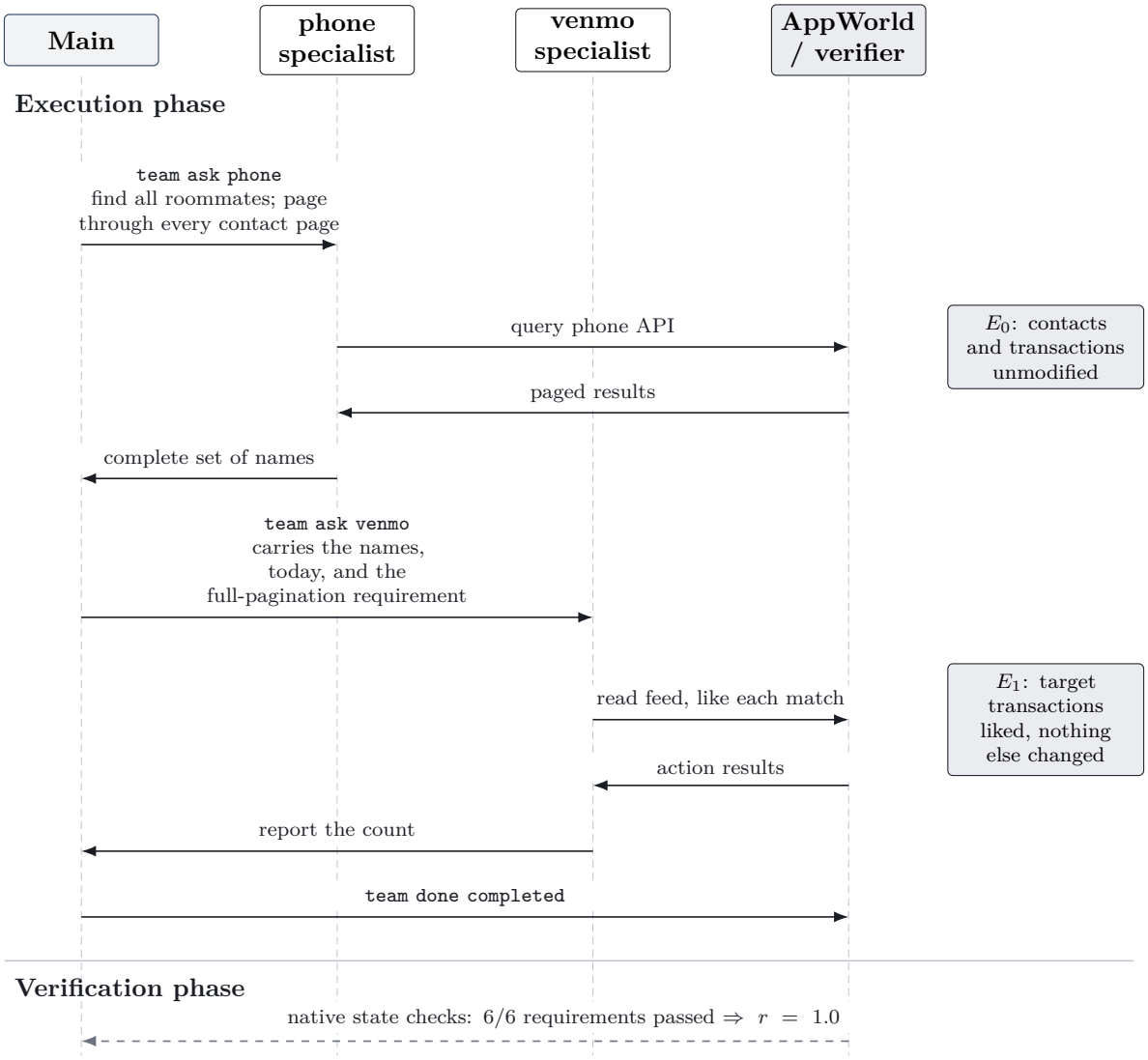

Figure 2: Information flow of the reference trajectory. Upstream roles only obtain facts; the Main's job is to organise those facts into a complete, executable request for the downstream role. The final reward comes from state E_1, not from matching the trajectory text. Dashed lines are the verification phase, after the agent has finished.

5.3 Reward and validity references

The verifier does not assess whether the conversation was "sensible", and it does not take the model's own account of its work. If the task has N state requirements of which N_{pass} pass, the reward is

$$r = \frac{N_{\text{pass}}}{N}.$$

This keeps partial credit while avoiding an LLM judge. The model may decompose and communicate however it likes; as long as the final state is correct with no extra side effects, it earns the same reward.

To confirm the reward means anything, each configuration needs three references. The first is the reference solution, proving the task is solvable. The second is the unconstrained control, giving a single agent every app, confirming the base task can be completed at all. The third is the do-nothing baseline, measuring how many checks already pass when no action is taken. Only when the constrained group sits between the solvable ceiling and the no-action floor, and produces a non-degenerate distribution across instances, is the task set suitable for RL.

6 Difficulty Control and Scaling

6.1 Which directions difficulty moves along

Difficulty varies along four directions. First, information visibility: the Main may read each role's capability documentation, or see only role names. Second, dependency structure: a task can grow from a two-step sequential dependency to three apps and multi-branch dependencies. Third, communication constraints, including the delegation budget and the star, chain and tree topologies. Fourth, the operational scale of the task itself — pages to traverse, entities involved, state modifications required.

These have to be added one layer at a time. A valid difficulty knob must satisfy two conditions: the reference solution still passes reliably, and the model's reward or failure structure changes reproducibly against the previous layer. A topology that merely makes the task unexecutable produces an environment fault, not higher difficulty — which is how chain got kept out of the delivery (§4.2).

6.2 Batch generation: the free half

The forging pipeline is four commands, and three of them call no model. `measure` reads the 147 ground-truth-bearing tasks and parses out which apps each one actually touched. `seams` runs a forward taint pass over the same reference solutions and keeps the 39 where facts genuinely cross apps. `render` turns the selected tasks and constraint configurations into Harbor task directories. All three are static analysis plus file writes, and finish on a laptop in minutes.

The reason they are free is written in the equation of §3: W acts only on the environment, and x and J are unchanged word for word. Multiplying along the configuration dimension therefore creates no new design work — one more visibility setting is one more pass of writing task directories. With 39 tasks and 2 implemented configurations, a single `render` command writes 78 task directories, plus an unconstrained control per task.

The real question is how many of those 78 directories can ship. The next two sections are the two costs, both downstream of `render`.

6.3 Cost one: reference solutions are currently hand-written

A Harbor task ships a `solution/` to prove it is solvable, and the reference solution here is not a script replay. The specialists are real LLMs, and the Main's job is to decompose rather than to execute, so a reference path records only the decomposition itself: ask phone for the names, then hand the names to venmo, with the specialists still doing the actual work. That makes this oracle probabilistic: verification needs live model calls and can fail on a bad specialist turn. That is a cost, stated rather than hidden. These paths are currently written one at a time, and `REFERENCE_PATHS` holds only 3 — a single task family. Running `render` over 39 tasks today writes 78 directories, of which **72 ship a `solve.sh` that fails loudly and says why**: a task with no reference solution cannot be proved solvable and therefore cannot enter a training set. This is the step standing furthest between here and "large batches".

But it can be automated, and the skeleton has already been computed. To measure selection yield at all, `seams.py` already has to answer "which app produced the fact, and which app's action consumed it". A reference path records that same triple: render (producer, fact, consumer) into (ask the producer, hand the fact to the consumer), and that is the whole structure of the path.

What remains by hand is not per-task authorship but per-app calling technique. Every brief, for instance, has to say "page through to the end" — that is phone's and venmo's pagination semantics, not a property of any one task. Such techniques count per app, not per task: 6 apps' worth in total, amortised across every task, which is a constant. This is measured, not hoped for — chasing down the "random" flake on `2a163ab_3` showed phone stopping at page one and dropping a coworker, not bad luck, and one added sentence about pagination fixed it for good (`reference.py` records the investigation). The shape of this cost is therefore: write the calling conventions for 6 apps once, then render per seam — not write 39 of them.

6.4 Cost two: verification, and it grows linearly in configurations

Verification cannot be free, because validity is a property of the rollout distribution rather than of a single draw (§4.4). A cell's yield needs `SEEDS_FOR_YIELD` = 5 replicates before it counts as measured, and the arithmetic follows directly: 39 tasks × 2 configurations × 5 seeds = 390 constrained rollouts, plus $39 \times 5 = 195$ controls, for 585 in total. At the per-run times the existing sweep measured (173 s mean across the two star settings, 30 s for the control), that is roughly 20 hours serial, or about 1.3 hours at 16-way parallelism.

So compute is not the wall. Two other things are. First, the real ceiling is usually API rate rather than compute: the same cells stretched from about 200 s to 6,700–15,000 s under rate limiting, a factor of 30–75 (`sweep/appworld_rate_limit_cost.json`). Second, this term grows linearly in the number of configurations — every new knob has to be re-run across every task with a full set of seeds before anyone knows whether that knob is difficulty or a fault. This is exactly the argument for measuring yield before scaling out: expanding on a configuration that has not been shown to produce a training signal expands invalid data.

6.5 The ceiling: this dimension holds only 13 independent task families

Even with both costs solved, the source itself has a hard ceiling, and it is far below 39.

The 39 seam-bearing tasks are not 39 independent samples: they are **13 task families of 3 variants each**. Variants share a dependency structure and differ only in entity, date and operational scale — the shipped `2a163ab_1/2/3` are three variants of one family, asking about roommates, siblings and coworkers respectively. By app combination the concentration is worse still (Table 5): 24 tasks across 8 families are phone+venmo.

Table 5: App-combination distribution of the 39 seam-bearing tasks (`sweep/appworld_seams.json`)

App combination	Tasks	Families
phone + venmo	24	8
phone + simple_note + venmo	3	1
simple_note + spotify	3	1
file_system + phone + venmo	3	1
phone + splitwise + venmo	3	1
file_system + simple_note	3	1
Total	**39**	**13**

One more layer tightens it further: AppWorld has 9 apps, but the 147 ground-truth-bearing tasks touch only 6 of them, and the seam-bearing tasks use only 6 combinations. So the trainable diversity on this dimension is 13 families and 6 combinations — not 39 tasks, and certainly not 78 directories. The abstract's "configuration count is not task count" needs one more turn of the screw here: **task count is not family count either**. To make the data volume actually grow, what is missing is not a knob. It is source tasks.

6.6 The admission test: what the method asks of a database

AppWorld is an argument to this method, not a premise of it. The method asks four things of a database, and dropping any one collapses it back into "design your own verifier" — which §4.1 argues is forbidden by the very reason the method exists:

1. **A programmatic verifier you did not write.** It is what makes the reward batch-computable, and what excuses you from ever defending a judgement about "how well they collaborated". Its value is precisely that somebody else already validated it.
2. **Real, executable, installable.** Not a description of a world. A database you cannot `pip install` or `docker pull` is one you will finish by building — and then you designed it, and

requirement 1 is gone with it.

3. **A reference solution per task.** It serves two purposes: deriving the roster (the roles come from the tools the reference solution touched, not from the designer's hand) and proving solvability. Only 147 of AppWorld's 750 tasks carry ground truth, and that is where the number 147 comes from.
4. **A tool boundary that can be cut.** The permission partition is the most fundamental of the four constraints (§3): if the tools cannot be split by role, the Main can bypass coordination and finish alone, and the other three become moot.

Admission is necessary and not sufficient. A database can clear all four and still yield nothing, because none of the four measures how *deep* the seams run: it may ship a flawless verifier and have every task collapse onto a single role. That is why seam analysis (§3.1) is a measurement rather than an assumption, and AppWorld nearly makes the point by itself — 96 of its 147 ground-truth tasks are single-app puzzles. The seams are also where substrate choice stops being free: SWE-smith has the strongest verifier on the list and scaled on the strength of requirement 1 [2], but its seams are code modules, so partitioning it yields a software-engineering task with extra agents rather than an orchestration task.

How far this generalises, stated honestly. Nothing in the method is AppWorld-shaped: the admission test is a property of a database, the roster is read off whatever a reference solution touches, the constraints act on an access surface, and the validity rule (§4.3) needs only a score, a floor and a control. But that is an argument from construction — checkable by reading it — and not a result. **This method has been run end to end on exactly one database.** The reach claimed here is therefore the admission test's: any database clearing all four requirements should work, and one has. A second substrate is the cheapest available falsification, and it has not been run; §8.1 treats that as a task rather than a caveat. The order for scaling up is: automate reference solutions out of the seams, measure yield on the 13 families that exist, and then widen sideways to new source databases — rather than piling more configurations onto the same task family.

7 Data Samples and Experimental Results

The minimal delivery sample is three instances from one task family. All three require obtaining a class of contacts from phone first, then having venmo act on the related transactions within a date range. They test the same capability structure but differ in entity, date and operational scale.

Table 6: Three Harbor data samples

Task ID	User task	Required coordination
`2a163ab_1`	Like all of today's transactions involving roommates on the Venmo social feed	phone finds the roommates' names; venmo pages today's feed and likes
`2a163ab_2`	Like all of yesterday's transactions involving siblings on the Venmo social feed	phone finds the siblings' names; venmo pages yesterday's feed and likes
`2a163ab_3`	Like all transactions involving coworkers from yesterday or today on the Venmo social feed	phone finds the coworkers' names; venmo pages across two dates and likes

All three samples' Harbor reference solutions pass all 6 state checks for a reward of 1.0. We then compare three configurations on the same tasks: open as the unconstrained control; stardocs, which forces the Main to delegate but lets it read the roles' capabilities; and star-names,

which further hides the capability docs and leaves only role names. The Main uses `gpt-5.6-sol` and the specialists `gpt-4.1`, with five seeds per cell — 15 rollouts per configuration.

Table 7: Constraint ablation results (3 tasks × 5 seeds = 15 rollouts per configuration)

Configuration	Mean reward	Three tasks (5 draws each)	Below control	Meaning
open	1.000	1.000/1.000/1.000	0/15	one agent holds every app
star-docs	0.855	0.833/0.833/0.900	**10/15**	delegation forced; the roles' real API catalogue is readable
star-names	0.445	0.833/0.167/0.333	12/15	delegation forced; role capabilities invisible
do-nothing	0.333	0.333/0.333/0.333	—	no action taken, but the answer type is right

Neither knob in this table is what its name says, so each needs taking apart.

star-docs drops 0.145 against open, and the five seeds are what made that sayable. The control scored 1.000 on *every one* of its 15 rollouts — zero variance — while star-docs landed below it on 10 of 15, spread across 0.667–1.000. A strong model still reaches the goal through a few delegations without direct tools; it simply pays about one requirement in six to do so. At one seed this figure read 0.056 and the honest caveat was that three tasks at one draw could not separate it from noise. That caveat was right, and the answer was seeds rather than a louder claim. More notable: the Main in this setting previously received the sentence "their capabilities are listed above" while *nothing* was above it but two role names. The thing that actually served the catalogue was `team docs` in the delivered container, while the measurement went down a different in-process path (see §5.1). After this round put the real API catalogues — 2,838 and 5,444 characters — in front of the Main, the score did not move at all: still 0.944. What moved was the delegation count, from 2–3 up to 3–5, and the time, from 100 s to 201 s. In other words, letting the Main see what the sub-agents can do bought more delegations and more latency, but no more success. Those four figures are all at seed 1, the only seed the earlier sweep has: the five-seed run re-prices the *partition*, not this comparison, and setting one draw against five would confound the docs knob with the sample size. The docs knob is unpriced, not free. Wall-clock deserves the same caution — across the five seeds star-docs averaged 201 s, 759 s, 2006 s, 482 s and 365 s, a tenfold spread on identical work, which is the organisation's TPM ceiling throttling the queue rather than the agent deliberating.

star-names' 0.855 → 0.445 looks at last like visibility biting. It is not. One of its three cells scores 0.167 on all five replicates — below the 0.333 "floor" — and a second sits there on four draws of five. They got there like this: the Main failed to find the target transactions and *honestly reported* that there were none matching, while AppWorld's `assert answers match` expects an action task's answer (`None`); prose does not match, so that requirement fails. Had the Main lied and claimed `completed`, it would have scored 0.333.

This does not have to be inferred from the sweep. It can be measured directly, with no model calls at all: open the same task, do nothing, vary only the submitted answer, and evaluate. `appworld_protocol_probe.py` does exactly that, with results in `sweep/appworld_protocol.json`.

Table 8: Same non-work, different words. Consistent across all three tasks, no model calls.

Agent	Score	Requirements lost
do nothing, claim `completed` falsely	**0.333**	4 state checks
do nothing, honestly report "found none"	**0.167**	the same 4, *plus* `assert answers match`

Both agents did exactly the same work (none), the terminal world state is identical, and the same four state checks fail. The only difference is *how they describe themselves*, and that difference is worth 1/6.

> **This reward pays** 1/6 **for a false claim of completion.** For a policy in training, that means lying about completion is strictly better than reporting failure honestly.

It also means the label "do-nothing = 0.333" in Table 7 is wrong: that is not a floor but the score for "got the answer type right while doing nothing", and an honest failure can land beneath it.

To be clear, **this method cannot fix it**. The verifier is AppWorld's, and changing it means starting to design verifiers — the one thing forbidden by the entire argument of §4.1 for why this method exists. What can be done is to measure it, report it, and — before training on this data — split the reward into a state term and a protocol term, recording them separately and driving the policy with the former only. The current $r = N_{\text{pass}}/N$ mixes the two into one scalar, and every difficulty curve in this report before this point is contaminated by it.

The current data is therefore **insufficient** to support the claim that hiding role capabilities is the more effective source of difficulty. Under the sandwich criterion of §4.4, star-names yields 0/3 usable cells, still below star-docs' 1/3.

8 Discussion and Extensions

8.1 Shore up the existing conclusions first

The current results remain a small-scale validation. The three instances come from one task family and each configuration has a single seed, so 0.056 and 0.167 cannot yet be treated as stable effects. The first next step is to add seeds on the existing tasks and give interval estimates for the differences; after that, add phone–simple_note, file_system–spotify and three-app tasks to confirm the conclusions hold across task types.

There is a second gap of a different kind, and it is cheaper to close than any of the above: **the method is presented as substrate-independent and has been run on one substrate** (§6.6). Seam analysis is the part that transfers most obviously and costs the least to check — it needs no rollouts and no model calls, only reference solutions — so running it against a second admitted database and publishing the seam distribution would either corroborate the generality claim or refute it for the price of an afternoon. Until somebody does, the claim rests on reading the construction rather than on evidence, and this report marks it as such rather than letting the survey of candidate databases do work that only a measurement can do.

Beyond that, the current reward reliably judges whether the final task was completed but cannot separate failures of decomposition, role selection and communication. That is also why this stage does not list Theory of Mind and dynamic recovery as separate dimensions. Finer-grained capability attribution can be done through paired probes or environment-side fault injection, but it should still be verified from the final state rather than falling back on scoring the conversation.

Once star-names' effect is stable across multiple seeds and task families, adding delegation budgets and chain/tree topologies becomes reasonable. That way each curriculum level builds on capabilities already mastered at the level below, and difficulty changes are easier to explain.

8.2 From spatial constraints to temporal ones

The current work changes the spatial structure of collaboration: who holds which tools, who knows what, along which topology messages may travel. The next stage can add a time axis to the same environment, so that information is not handed over all at once at the start but appears when the agent asks for it or when the task reaches a particular state. That would train hidden-requirement discovery and dynamic replanning under user intervention, without giving up terminal-state verification.

A minimal version generates paired samples from the same complete task. The control group still receives the full instruction; the experimental group receives a deliberately incomplete but realistically phrased request, with the contact scope, date or compatibility preference held in hidden user state in the sidecar. The Main can obtain those constraints only by noticing the gap and asking the user. Both groups share an initial world and a final verifier, so there is no need to score "was that a good question?" separately: if the key question never gets asked, the agent cannot reliably reach the correct terminal state; and asking about everything burns a limited interaction budget.

User intervention can serve as the next difficulty layer. After the agent has done part of the work, simulate the user adding a changed scope or a new restriction, requiring the Main to update the plan, keep the results that remain valid, and pass the new constraint to the relevant roles. Two cases must be distinguished here: if the added information was always part of the original goal and merely unstated, the final judge can stay unchanged; if the user genuinely changed the goal, the new terminal state needs an equally trustworthy environment check, and cannot be judged from the conversation alone.

This temporal extension can be written as

$$\tau_{c,h} = (x_0, \mathcal{E}_c, U_h, \Gamma_h, J),$$

where U_h is hidden user state existing only in the sidecar and Γ_h decides when information is asked for, revealed or modified. The project's most important boundary still has to hold: hidden state must never enter the Main's filesystem, the interaction record must not be forgeable by the agent, and the final reward must continue to come from replayable environment state. Only once paired experiments reliably separate the scores of full instruction, hidden requirement and mid-run intervention should it enter a formal curriculum.

9 Conclusion

The key to this method is not inventing new tasks out of nothing. It is finding coordination dependencies inside real tasks that already exist, then making those dependencies explicit through permission and information allocation. The Main must plan and delegate, the sub-agents can only do local work, and the final reward still comes from AppWorld's real world state.

Configurations can therefore be generated in bulk and the verification logic reused. But **whether difficulty can be tuned along these directions is not something this report has proved**. What the three samples prove is that the partitioned environment and the Harbor verifier run correctly, and that reference solutions pass all 6 state checks. Beyond that, the existing sweep observed a non-degenerate training signal in exactly one star-docs instance; star-names is not an established difficulty increment (§7), the chain topology is not built, and parallel scheduling is out of scope.

What comes next is therefore not scaling up but making this one cell solid: add seeds to the shipped configurations, strip the submission protocol's contamination out of the reward, and extend the delivered samples from one task family to different dependency structures. Hidden user requirements and mid-run intervention are worthwhile extensions, but they should sit on top of a base configuration already proven to produce a training signal reliably — and no such configuration exists yet.

References

[1] Harsh Trivedi et al. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. *ACL 2024*, arXiv:2407.18901. https://arxiv.org/abs/2407.18901

[2] John Yang, Kilian Lieret, Carlos E. Jimenez, et al. SWE-smith: Scaling Data for Software Engineering Agents. arXiv:2504.21798, 2025. https://arxiv.org/abs/2504.21798

[3] Kunlun Zhu, Hongyi Du, Zhaochen Hong, et al. MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents. *ACL 2025*, arXiv:2503.01935. https://arxiv.org/abs/2503.01935

[4] Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657, 2025. https://arxiv.org/abs/2503.13657

[5] Harbor Framework. Task Structure: Creating and Running Tasks for Agentic Environments. Documentation, accessed 2026-07-16. https://www.harborframework.com/docs/tasks